

Mesure de distance caméra-visage en temps réel

Détection de visage et estimation de distance avec OpenCV

Rattrapage de Machine Learning

8 août 2026

Table des matières

1	Objectif du projet	2
2	Principe théorique	2
2.1	Le modèle du sténopé (pinhole camera)	2
2.2	Étape de calibration	2
2.3	Schéma du principe	2
3	Description du code	3
3.1	Le détecteur de visages (Haar Cascade)	3
3.2	Calcul de la distance focale	3
3.3	Calcul de la distance en temps réel	3
3.4	Extraction de la largeur du visage	3
3.5	Boucle principale	4
4	Mise en œuvre pratique et difficultés rencontrées	4
4.1	Correction d'un bug dans le code fourni	4
4.2	Incompatibilité de version d'OpenCV	4
4.3	Image de référence manquante	5
4.4	Utilisation de la caméra d'un téléphone	5
4.5	Ajustement des paramètres de détection	5
5	Résultats	5
6	Conclusion	5

1 Objectif du projet

L'objectif de ce projet est de mesurer, en temps réel, la distance entre une caméra (webcam d'ordinateur ou caméra de téléphone) et le visage d'une personne, en utilisant uniquement une image 2D — sans capteur de profondeur, sans LiDAR, sans caméra stéréo.

Le principe repose sur une propriété géométrique simple : **plus un objet est loin de la caméra, plus il paraît petit dans l'image**. En connaissant la taille réelle d'un visage humain et en mesurant sa taille apparente en pixels, on peut en déduire la distance réelle par une simple règle de proportionnalité, une fois la caméra "calibrée".

2 Principe théorique

2.1 Le modèle du sténopé (pinhole camera)

Une caméra peut être modélisée simplement par un modèle de *sténopé* : tous les rayons lumineux passent par un point unique (le centre optique) avant de former une image sur le capteur. Ce modèle établit une relation de proportionnalité inverse entre :

- la taille réelle d'un objet (W , en centimètres),
- la distance entre l'objet et la caméra (D , en centimètres),
- la taille apparente de l'objet sur l'image (w , en pixels),
- la distance focale de la caméra (f , en pixels).

La relation fondamentale est :

$$f = \frac{w \times D}{W} \quad (1)$$

Une fois f connue (calibrée une seule fois), on peut retrouver la distance D' pour n'importe quelle nouvelle largeur apparente w' :

$$D' = \frac{W \times f}{w'} \quad (2)$$

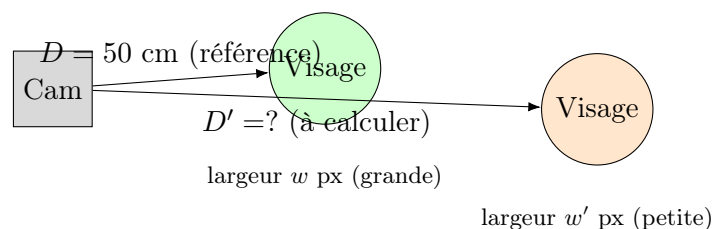
2.2 Étape de calibration

La calibration consiste à prendre **une seule photo de référence**, dans laquelle :

- la distance réelle entre le visage et la caméra est **connue** (fixée à $D = 50$ cm dans ce projet),
- la largeur réelle moyenne d'un visage humain est prise comme $W = 14,3$ cm (valeur anthropométrique standard).

On mesure alors la largeur du visage en pixels sur cette photo (w), ce qui permet de calculer f grâce à l'équation (1). Cette valeur f dépend uniquement de la caméra utilisée (elle change si on change de caméra, d'où la nécessité de recalibrer pour chaque appareil).

2.3 Schéma du principe



Plus le visage est loin, plus sa largeur apparente en pixels diminue — c’est cette diminution que le script mesure image par image.

3 Description du code

Le projet est composé de deux fichiers fournis initialement :

- `Distance_RealTIME.py` : le script principal ;
- `haarcascade_frontalface_default.xml` : un classifieur pré-entraîné pour la détection de visages (fourni par la bibliothèque OpenCV).

3.1 Le détecteur de visages (Haar Cascade)

La détection de visage utilise un **classifieur en cascade de Haar**, une technique de vision par ordinateur antérieure au deep learning, encore largement utilisée pour sa rapidité. Le fichier XML contient les paramètres d’un modèle entraîné sur des milliers d’images positives (contenant des visages) et négatives (sans visage), capable de repérer des motifs de contraste caractéristiques d’un visage (yeux, nez, bouche).

```
1 face_detector = cv2.CascadeClassifier("haarcascade_frontalface_default.xml")
```

3.2 Calcul de la distance focale

```
1 def Focal_Length_Finder(measured_distance, real_width, width_in_rf_image):
2     focal_length = (width_in_rf_image * measured_distance) / real_width
3     return focal_length
```

Cette fonction implémente directement l’équation (1) : elle multiplie la largeur en pixels de l’image de référence par la distance connue (50 cm), puis divise par la largeur réelle du visage (14,3 cm).

3.3 Calcul de la distance en temps réel

```
1 def Distance_finder(Focal_Length, real_face_width, face_width_in_frame):
2     distance = (real_face_width * Focal_Length) / face_width_in_frame
3     return distance
```

Cette fonction implémente l’équation (2) : pour chaque nouvelle image (chaque *frame*), connaissant la focale calibrée et la largeur du visage détecté en pixels, on retrouve la distance réelle.

3.4 Extraction de la largeur du visage

```
1 def face_data(image):
2     face_width = 0
3     gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
4     faces = face_detector.detectMultiScale(gray_image, 1.3, 5)
5
6     for (x, y, w, h) in faces:
7         cv2.rectangle(image, (x, y), (x+w, y+h), GREEN, 2)
8         face_width = w
9
10    return face_width
```

Cette fonction :

1. convertit l'image en niveaux de gris (le détecteur Haar ne travaille pas sur la couleur) ;
2. appelle `detectMultiScale`, qui renvoie une liste de rectangles (`x`, `y`, `largeur`, `hauteur`) correspondant aux visages trouvés ;
3. dessine un rectangle vert autour du (ou des) visage(s) détecté(s) ;
4. renvoie la largeur en pixels du visage, utilisée ensuite pour le calcul de distance.

Les deux paramètres de `detectMultiScale` sont importants :

- `scaleFactor` (1.3 ou 1.1) : réduit l'image par ce facteur à chaque échelle testée. Une valeur plus proche de 1 (ex. 1.1) rend la détection plus précise mais plus lente et plus sensible aux faux positifs.
- `minNeighbors` (5 ou 3) : nombre minimum de détections voisines nécessaires pour valider un visage. Une valeur plus basse détecte plus facilement mais génère plus de fausses détections.

3.5 Boucle principale

```

1 cap = cv2.VideoCapture(0) # ou une URL IP pour une camera de telephone
2
3 while True:
4     _, frame = cap.read()
5     face_width_in_frame = face_data(frame)
6
7     if face_width_in_frame != 0:
8         Distance = Distance_finder(Focal_length_found, Known_width,
9                                     face_width_in_frame)
10        cv2.putText(frame, f"Distance: {round(Distance,2)} CM", (30, 35), fonts,
11                        0.6, GREEN, 2)
12
13    cv2.imshow("frame", frame)
14    if cv2.waitKey(1) == ord("q"):
15        break

```

Cette boucle tourne indéfiniment : elle lit une image de la caméra, détecte le visage, calcule et affiche la distance, jusqu'à ce que la touche q soit pressée.

4 Mise en œuvre pratique et difficultés rencontrées

4.1 Correction d'un bug dans le code fourni

Le code original contenait une inversion :

```

1 for (x, y, h, w) in faces: # incorrect : h et w inverses

```

alors que `detectMultiScale` renvoie systématiquement l'ordre (`x`, `y`, `w`, `h`). Cette inversion faussait le calcul de la largeur du visage. Correction :

```

1 for (x, y, w, h) in faces: # ordre correct

```

4.2 Incompatibilité de version d'OpenCV

Lors de l'installation, la commande `pip install opencv-python` installait par défaut la toute dernière version (5.0.0.93), qui présentait un bug empêchant l'accès à `cv2.CascadeClassifier` (`AttributeError`). La solution a été de fixer une version stable et éprouvée :

```

1 pip install "opencv-python==4.10.0.84"

```

4.3 Image de référence manquante

Le script nécessite un fichier `Ref_image.png`, non fourni dans le projet, correspondant à la photo de calibration prise à exactement 50 cm. Cette image a été capturée directement en ligne de commande via OpenCV, en laissant 5 secondes à la personne pour se positionner avant la capture automatique.

4.4 Utilisation de la caméra d'un téléphone

Pour utiliser la caméra d'un smartphone plutôt que la webcam de l'ordinateur, l'application **IP Webcam** a été installée sur le téléphone. Une fois le serveur lancé, l'application fournit une adresse réseau locale (ex. `http://10.16.207.99:8080`), accessible depuis n'importe quel appareil connecté au même réseau Wi-Fi. Le flux vidéo est alors récupéré en remplaçant l'index de la webcam par l'URL du flux :

```
1 cap = cv2.VideoCapture("http://10.16.207.99:8080/video")
```

Point important : la distance focale calculée dépend entièrement de la caméra utilisée. Changer de caméra (PC vers téléphone) impose de **recalibrer**, c'est-à-dire reprendre une nouvelle image `Ref_image.png` avec cette même caméra.

4.5 Ajustement des paramètres de détection

La caméra du téléphone (résolution et angle différents de la webcam du PC) nécessitait des paramètres de détection légèrement plus souples pour repérer correctement le visage :

```
1 faces = face_detector.detectMultiScale(gray_image, 1.1, 3)
```

5 Résultats

Après calibration, le système affiche en continu, dans la fenêtre vidéo, un encadré indiquant la distance mesurée en centimètres, par exemple :

Distance: 51.09 CM

Les mesures obtenues sont cohérentes avec la distance réelle mesurée physiquement (autour de 50 cm), ce qui valide le bon fonctionnement du système de calibration et de mesure.

Limite observée : en présence de plusieurs visages dans le champ de la caméra (ex. salle de classe), le script peut confondre le visage cible avec un autre, provoquant des valeurs de distance erratiques. Cela s'explique par le fait que la fonction `face_data` ne garde que le *dernier* visage détecté dans la boucle, sans logique de sélection du visage principal.

6 Conclusion

Ce projet illustre une méthode simple et peu coûteuse d'estimation de distance à partir d'une seule caméra 2D, en combinant :

- la détection d'objets par classifieur en cascade de Haar (OpenCV) ;
- un modèle géométrique de caméra (sténopé) pour convertir une taille en pixels en distance réelle.

Cette approche, bien que moins précise qu'un capteur de profondeur dédié, reste efficace pour des applications simples (respect de distance sociale, cadrage automatique, systèmes d'alerte de proximité) et ne nécessite aucun matériel spécialisé au-delà d'une caméra standard.